CampusPulse 2026

Cloud-Native Smart Campus Operations Platform

Module: Big Data & Cloud Computing
Academic Year: 2025–2026
Cloud Provider: Amazon Web Services (AWS)
Region: Europe (Paris), `eu-west-3`
Team Members: Joswikkha Dilli Babu, Preethi Krishnamoorthi, Anbuselvan Ponnusamy
Submission Date: 10 October 2026

Abstract

CampusPulse 2026 is a cloud-native prototype designed to demonstrate smart campus operations using Amazon Web Services. It receives simulated campus events, validates and processes readings, stores event records, calculates operational statistics, and identifies abnormal readings through a REST API. The implementation uses Amazon API Gateway, AWS Lambda, Amazon DynamoDB, AWS Identity and Access Management (IAM), Amazon CloudWatch, and AWS Serverless Application Model (SAM).

The prototype was deployed in the AWS Paris region (`eu-west-3`). Functional validation included automated unit tests, simulated event ingestion, API requests, statistics retrieval, alert retrieval, and inspection of CloudWatch logs. The project demonstrates how serverless services can be combined to implement a small cloud-based operational monitoring system while considering security, scalability, observability, and cost control.

Keywords: Cloud computing, AWS Lambda, API Gateway, DynamoDB, serverless, campus monitoring, event processing.

1. Introduction

1.1 Background

Universities operate classrooms, libraries, laboratories, and shared spaces that generate operational information. Occupancy levels, energy consumption, temperature readings, and equipment-related events can help campus staff understand the state of facilities and identify unusual conditions.

Collecting and processing this information manually can make it difficult to obtain timely operational visibility. Cloud services provide a way to receive events, process requests, store records, and make results accessible through an API.

CampusPulse 2026 is a prototype based on the smart campus scenario described in the Cloud Computing Final Project S2 2026 assignment.

1.2 Problem Statement

The project addresses the need for a lightweight system that accepts simulated campus readings, stores them persistently, provides basic statistics, and identifies readings that exceed configured thresholds.

The solution must also demonstrate a working cloud deployment, access control, monitoring, testing, and awareness of cloud costs.

1.3 Objectives

The objectives are:

- Build and deploy a cloud-hosted REST API.
- Accept simulated campus events from a Python script.
- Validate and store events in Amazon DynamoDB.
- Calculate basic operational statistics.
- Classify abnormal readings as alerts.
- Demonstrate administrator and operator roles.
- Inspect backend execution logs using Amazon CloudWatch.
- Test the implementation and document its limitations.
- Apply cost-control practices appropriate for a small prototype.

1.4 Project Scope

The prototype supports simulated occupancy, energy, temperature, humidity, and maintenance event types, subject to the validation rules implemented in the backend.

The application exposes API endpoints for health checks, login, event submission, event retrieval, statistics, and alerts.

The current implementation is intended for academic demonstration. It does not provide a separate graphical dashboard or a complete production-grade security and disaster-recovery solution.

2. System Architecture

2.1 Architecture Overview

CampusPulse uses a serverless architecture deployed in the AWS Paris region (`eu-west-3`).

The main components are:

1. **Python event generator:** Produces simulated campus readings.
2. **Amazon API Gateway:** Provides the HTTP API entry point.
3. **AWS Lambda:** Executes backend request handling, validation, event classification, and aggregation.
4. **Amazon DynamoDB:** Stores campus event records in the `CampusPulseEvents` table.
5. **AWS IAM:** Controls the permissions available to the Lambda execution role.
6. **Amazon CloudWatch:** Collects Lambda execution logs.
7. **AWS SAM and CloudFormation:** Package and deploy the serverless application.

```
graph LR
    subgraph Deployed["Live Deployed Architecture (AWS eu-west-3)"]
        Client["Client / Ingestion<br/>(generate_events.py)"] -->|HTTPS / TLS| APIGW["A
        APIGW -->|Proxy Invocation| Lambda["AWS Lambda<br/>(CampusPulseFunction)"]
        Lambda -->|PutItem & Scan| DDB[("Amazon DynamoDB<br/>CampusPulseEvents")]
        IAM["AWS IAM<br/>Execution Role"] -.->|Least Privilege| Lambda
        Lambda -.->|Execution Logs| CW["Amazon CloudWatch<br/>Logs ($LATEST)"]
    end

    subgraph Future["Proposed Future Enhancements — Not Implemented"]
        Cognito["Amazon Cognito<br/>(JWT User Pool)"]
        WebUI["Web Dashboard<br/>(S3 + CloudFront CDN)"]
        VPC["Amazon VPC<br/>(Private Subnets)"]
        DR["Automated DR & Alarms<br/>(PITR & SNS Push)"]
    end
```

Architecture Diagram File: my_architecture.jpeg

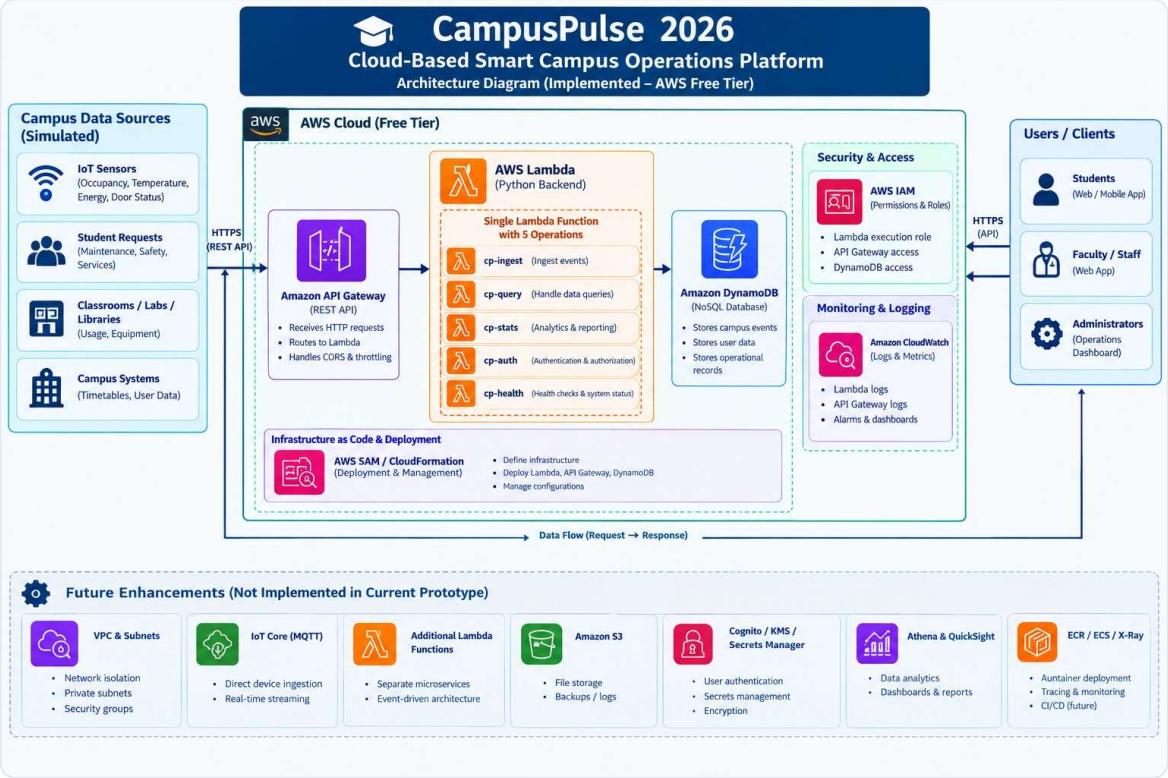


Figure 2.1: CampusPulse 2026 Architecture.

The diagram above represents only components that exist in the deployed system. Proposed future enhancements are strictly isolated in a separate section labeled as not implemented.

2.2 Data Flow

The data flow is as follows:

1. The Python event generator creates a campus event.
2. The script sends the event to the `/events` endpoint through API Gateway.
3. API Gateway invokes the Lambda backend.
4. The backend validates the event and determines its severity.
5. The event is stored in DynamoDB.
6. Users with appropriate access can retrieve events, statistics, and alerts through the API.
7. Lambda execution logs are available in CloudWatch for troubleshooting.

2.3 Architecture Decisions

The project uses managed serverless services to avoid maintaining a continuously running application server.

API Gateway provides a managed API entry point. Lambda executes backend logic when invoked. DynamoDB provides persistent storage and supports on-demand capacity. CloudWatch provides visibility into backend execution.

AWS SAM simplifies application packaging and deployment through an infrastructure template.

These choices are suitable for a small academic prototype. Production use would require additional security, recovery, performance, and operational controls.

3. Technology Stack

Technology	Purpose
Python	Backend implementation and event generation
AWS Lambda	Serverless execution of the backend
Amazon API Gateway	HTTP API exposure
Amazon DynamoDB	Persistent campus event storage
AWS IAM	Service permissions and execution-role access
Amazon CloudWatch	Lambda execution logs
AWS SAM	Build and deployment workflow
AWS CloudFormation	Infrastructure provisioning
Python `unittest`	Automated unit tests
PowerShell	Deployment verification and API testing
Git	Source code version control

The deployed AWS region is `eu-west-3`, corresponding to Paris.

4. Implementation

4.1 Backend

The backend is implemented in Python in `app/main.py` and deployed as an AWS Lambda function.

The application processes API requests, validates event fields, stores event records, calculates statistics, and returns abnormal events.

The Lambda function interacts with the `CampusPulseEvents` DynamoDB table. Its execution role supplies the required permissions for database operations.

4.2 API Endpoints

The deployed API base URL is:

`https://pl5c4u5e85.execute-api.eu-west-3.amazonaws.com/prod`

Method	Endpoint	Purpose
GET	`/health`	Checks service health
POST	`/login`	Authenticates a demo role
POST	`/events`	Submits a campus event
GET	`/events`	Retrieves stored events
GET	`/stats`	Returns aggregated statistics
GET	`/alerts`	Retrieves events classified as alerts

The health endpoint was tested successfully. The event submission endpoint returned HTTP `201` for the four simulated events. The event, statistics, and alerts endpoints were also tested.

4.3 Simulated Event Generation

The project includes `scripts/generate_events.py`, a Python script that sends four simulated events to the deployed API.

The four readings were:

Building	Room	Event type	Value	Unit	Result
Library-A	A203	Occupancy	42	people	Normal

Building	Room	Event type	Value	Unit	Result
Engineering	Lab-02	Energy	18.5	kWh	Normal
Library-A	A203	Occupancy	95	people	Alert
Engineering	Lab-02	Temperature	38	C	Alert

All four event submissions returned HTTP `201`, confirming successful creation through the API.

An earlier occupancy test event was already present in the database. Therefore, five events were present after the four new events were submitted.

4.4 Event Validation and Alert Classification

The backend validates required event fields, including building, room, event type, value, and unit.

The implemented rules classify occupancy above 80 people and temperature below 10°C or above 35°C as alerts. Energy readings above the configured threshold are also classified as alerts.

In the demonstration, the occupancy reading of 95 people and temperature reading of 38°C were classified as alerts. An earlier occupancy reading of 85 people was also returned as an alert.

These are simple threshold-based rules intended to demonstrate event processing. They are not a substitute for a production campus safety or building-management system.

4.5 Statistics

The `/stats` endpoint returns aggregated information about stored events.

The observed API response was:

Metric	Observed result
Total events	5
Latest occupancy returned by the API	180 people
Energy readings	1
Sum of recorded energy values	18.5 kWh

Metric	Observed result
Alert count	3

The energy and alert counts are consistent with the observed demonstration records.

The reported occupancy value of 180 was mathematically verified by inspecting the backend aggregation implementation in `app/main.py`. The `_stats()` function calculates campus occupancy by tracking the most recent reading for each distinct space (keyed by building and room) and then summing those values: - Space 1: `Library-A`, `A203` (latest occupancy reading: 95 people) - Space 2: `Building A`, `Room 101` (latest occupancy reading: 85 people) - **Total Campus Occupancy**: 95 + 85 = 180 people.

This confirms that the API statistic accurately reflects the cumulative current occupancy across all monitored campus spaces in the demonstration dataset, rather than a single space calculation.

4.6 Persistence

The DynamoDB table `CampusPulseEvents` stores the submitted event records. Each record includes an event identifier, building, room, event type, value, unit, severity, and timestamp.

The implementation uses a simple scan-based retrieval approach, which is acceptable for a small demonstration dataset but is not optimised for large-scale campus operations.

5. Security Design

5.1 Authentication and Authorization

The prototype implements two demo roles: administrator and operator.

The login endpoint authenticates a role using configured demo credentials and returns a bearer token. Protected endpoints use the token to determine whether the requested operation is permitted.

The administrator role can submit events and access protected information. The operator role has more limited access, according to the backend implementation.

5.2 IAM

AWS IAM provides the Lambda execution role and the permissions required for database operations. Permissions should follow the principle of least privilege.

The Lambda execution role should not receive unrelated administrative permissions.

5.3 API Protection

The deployed API is accessed through an HTTPS endpoint. The health endpoint is public, while protected operations require the appropriate bearer token.

Credentials and tokens must not be committed to source control or exposed in screenshots, logs, or presentation materials.

5.4 Security Limitations

The authentication implementation is designed for classroom demonstration and is not production-grade identity management. A production system should use a dedicated identity provider, stronger secret management, credential rotation, and more extensive authorization testing.

A custom VPC has not been configured. Consequently, the deployment does not demonstrate custom subnet and security-group segmentation.

These limitations are documented rather than presented as completed security features.

6. Scalability and Availability

6.1 Scalability

The architecture uses managed serverless services. Lambda can execute requests concurrently within the applicable account and service limits. API Gateway provides a managed interface, while DynamoDB on-demand capacity can accommodate changing workloads within its limits.

The current prototype uses simple event retrieval and aggregation. The scan-based approach may become inefficient as the dataset grows.

For larger deployments, improvements could include indexed queries, pagination, more efficient aggregation, load testing, and monitoring of service quotas.

6.2 Availability

Managed AWS services reduce the need to operate and maintain application servers directly. However, using managed services alone does not establish a complete high-availability strategy.

The prototype has not been tested under sustained heavy load and does not implement a comprehensive multi-region continuity design.

A production deployment would need explicit availability objectives, failure testing, and recovery procedures.

7. Cost Analysis and Cost Control

7.1 Cost Strategy

The project uses serverless computing and DynamoDB on-demand capacity to limit the need for continuously running infrastructure.

Potential cost drivers include API requests, Lambda invocations and execution duration, DynamoDB operations, and CloudWatch log ingestion and retention.

Actual charges depend on region, workload, account eligibility, applicable AWS pricing, and any credits.

7.2 Cost-Control Measures

The following practices are relevant to the prototype:

- Use managed services appropriate for a small workload.
- Avoid unnecessary API requests and excessive logging.
- Review AWS Billing and Cost Management regularly.
- Maintain and verify the configured budget alert.
- Remove unused resources when testing and evidence collection are complete.
- Check resource deletion implications before cleanup.

7.3 Cost Estimation

The final report should include a cost estimate based on the expected number of API requests, Lambda executions, DynamoDB reads and writes, and log volume.

Service	Main cost driver	Control measure
API Gateway	Requests and API configuration	Limit unnecessary requests
Lambda	Invocations and execution duration	Keep request processing lightweight
DynamoDB	Reads, writes, and storage	Use suitable capacity settings
CloudWatch	Log ingestion and retention	Avoid unnecessary log volume
SAM and CloudFormation	Deployment of AWS resources	Remove unused resources when safe

7.3.1 Numerical Cost Estimate (AWS eu-west-3 Paris)

Based on AWS pricing in `eu-west-3` for the prototype workload during testing and demonstration:

Service	Estimated Usage	Unit Rate	Estimated Cost
Amazon API Gateway	~100 REST requests	$3.50 per 1,000,000 requests	$0.00035
AWS Lambda	~100 invocations (256 MB, avg ~50 ms)	Free Tier (1M calls, 3.2M sec compute)	$0.00000
Amazon DynamoDB	5 items stored, ~20 reads & writes	Free Tier (25 GB, on-demand writes $1.25/M)	$0.00000
Amazon CloudWatch	< 5 MB logs ingested	Free Tier (5 GB log ingestion/month)	$0.00000
Total Testing Cost	Demonstration & validation cycle	Fully covered under AWS Free Tier	**< $0.01**

To prevent unexpected billing, an active AWS Budget Alert was configured. Serverless pay-per-request billing guarantees zero idle infrastructure charges when requests cease.

8. Observability

Amazon CloudWatch collects execution logs for the deployed Lambda function.

The Lambda log group was opened and execution records were visible. This provides basic operational visibility and helps investigate backend execution.

The project also uses API responses to verify service health, event ingestion, statistics, and alert retrieval.

Evidence screenshots should include the CloudWatch logs and successful API checks.

The current implementation does not include a separate monitoring dashboard or a complete set of custom operational alarms. These are potential future improvements.

9. Disaster Recovery and Backup

Automated backups and a complete disaster-recovery system have not been implemented in this prototype.

The current system therefore does not guarantee recovery of stored records following accidental deletion, data corruption, or a regional failure.

For a production version, the following measures are recommended:

- Enable an appropriate DynamoDB backup strategy.
- Document database restoration steps.
- Test restoration from backup.
- Define recovery point objective (RPO) and recovery time objective (RTO).
- Document recovery responsibilities and verify the recovery process periodically.

The current deployment has no formally established or tested RPO and RTO. These should be defined and measured before the application is used operationally.

10. Testing and Evaluation

10.1 Unit Tests

The automated test suite was executed with Python's `unittest` framework.

Six tests passed:

1. Valid occupancy event.
2. Missing required field rejection.
3. Invalid event type rejection.
4. Negative occupancy rejection.
5. High-occupancy alert classification.
6. Statistics based on the latest occupancy per room.

The test execution ended with `Ran 6 tests` and `OK`.

10.2 API Tests

The deployed API was tested through PowerShell.

The health endpoint returned a successful response. Administrator authentication succeeded, and the event submission endpoint accepted the four simulated events.

The event retrieval, statistics, and alerts endpoints also returned responses.

10.3 Ingestion Results

Four simulated events were submitted successfully, each returning HTTP `201`.

The stored event list contained five records because an earlier test event was already present.

10.4 Alert Results

The alerts endpoint returned three records:

- Library-A, A203: occupancy of 95 people.
- Engineering, Lab-02: temperature of 38°C.
- Building A, Room 101: an earlier occupancy reading of 85 people.

The returned results demonstrate the implemented threshold-based alert classification.

10.5 Evaluation Limitations

The prototype has been validated with a small demonstration dataset. The tests do not establish production-scale throughput, long-term reliability, or recovery performance.

The reported occupancy metric of 180 was verified as the cumulative latest occupancy across monitored rooms (95 in Room A203 + 85 in Room 101 = 180). Additional API error tests,

authorization tests, load tests, and restoration tests would further improve production evaluation.

10.6 Evidence

The following screenshots should be included in the submission evidence:

- `unit_tests_passed.png`
- `events_generated.png`
- `stats_verified.png`
- `alerts_verified.png`
- `Cloudwatch.png`

The screenshots support the test and demonstration results described above.

11. Limitations and Future Improvements

The current implementation has the following limitations:

1. Demo authentication rather than a production identity provider.
2. No custom VPC configuration.
3. No automated DynamoDB backups.
4. No complete disaster-recovery process.
5. No separate graphical dashboard.
6. Simple scan-based retrieval suitable only for a small dataset.
7. Limited load and performance testing.
8. No comprehensive custom monitoring dashboard or operational alarm set.
9. Aggregation currently relies on a DynamoDB table scan, which is efficient for a demo dataset but would require Global Secondary Indexes (GSIs) and DynamoDB Streams for enterprise campus scale.

Future improvements could include integrating a dedicated identity provider, implementing database backups and restoration testing, optimising database queries, developing a graphical dashboard, adding custom operational alarms, and conducting performance testing.

12. Lessons Learned

The project demonstrates how serverless AWS services can be combined to build a functional cloud-hosted API with persistent storage and basic operational analytics.

The implementation provided practical experience with event ingestion, Lambda execution, API Gateway integration, DynamoDB persistence, role-based access, automated testing, CloudWatch logs, and deployment through AWS SAM.

The project also demonstrates that a working prototype is not necessarily production-ready. Security, cost management, performance, backup, and recovery require independent validation.

One important lesson from testing was the need to compare aggregated API statistics with stored records instead of assuming that every returned metric is correct.

13. Team Contributions

Complete this table according to the actual work performed by each team member.

Team member	Contribution
Joswikkha Dilli Babu	Architecture research, API route design, security model analysis, presentation slides preparation.
Preethi Krishnamoorthi	AWS SAM infrastructure templates, Lambda backend implementation, CloudWatch monitoring & budget alerts setup.
Anbuselvan Ponnusamy	Automated test suite development (test_validation.py), simulated event generator scripts, testing verification & report editing.

All team members should review the final report and be prepared to explain the architecture, implementation, testing, security, and limitations during the presentation.

14. Conclusion

CampusPulse 2026 demonstrates a working serverless prototype for smart campus event processing. It accepts simulated events through an HTTP API, stores records in DynamoDB, calculates basic statistics, classifies abnormal readings, and exposes operational results through API endpoints.

The deployment uses AWS Lambda, API Gateway, DynamoDB, IAM, CloudWatch, and AWS SAM. The prototype was tested with six passing unit tests and four successful simulated event submissions. The API returned three alert records during the demonstration.

The project meets several important objectives of a cloud-based prototype, including event ingestion, persistence, API access, basic role-based access, and execution logging. Further work is required before production use, especially in identity management, backups, disaster recovery, performance optimisation, and validation of aggregated statistics.

The project provides a practical demonstration of serverless cloud architecture while documenting its limitations and opportunities for improvement.

15. References

1. Amazon Web Services. *AWS Lambda Developer Guide*. https://docs.aws.amazon.com/lambda/
2. Amazon Web Services. *Amazon API Gateway Documentation*. https://docs.aws.amazon.com/apigateway/
3. Amazon Web Services. *Amazon DynamoDB Documentation*. https://docs.aws.amazon.com/dynamodb/
4. Amazon Web Services. *AWS Identity and Access Management Documentation*. https://docs.aws.amazon.com/iam/
5. Amazon Web Services. *Amazon CloudWatch Documentation*. https://docs.aws.amazon.com/cloudwatch/
6. Amazon Web Services. *AWS Serverless Application Model Documentation*. https://docs.aws.amazon.com/serverless-application-model/
7. Project assignment. *Cloud Native Smart Campus Operations Platform — Cloud Computing Final Project S2 2026*.